

가천대학교 스마트시티학과 – 가상공간과 메타버스

Lecture 19 - Validation and Final Integration Lab

Chris Ohk
utilForever@gmail.com

학습 목표

- Lecture 18의 검증과 반복 실험을 기존 도시 프로그램에 적용한다.
- 함수 6개를 작성해 계산 결과를 확인하고 실험 결과를 비교한다.
  - 작성 → 단계 검사 → 화면 확인 → 조건을 바꿔 실험

실행 준비

- 압축을 푼 스켈레톤 폴더에서 프로그램을 실행한다.
  - Windows: `py -m tkinter` → `py app.py`
  - macOS / Linux: `python3 -m tkinter` → `python3 app.py`
- `validation_rules.py`를 저장한 뒤 앱을 닫고 다시 실행한다.

역할 나누기

- `validation_rules.py`의 TODO 1~6을 채운다.

파일	역할	수업 중 수정
validation_rules.py	검증 / 실행 / 결과 비교	직접 작성
validation_gui.py	버튼 / 표 / 그래프	제공
validation_support.py	기록 저장 / 불러오기	제공
checks_validation.py	단계 검사 / 통합 검사	제공
기존 도시 코드	도로 / 서비스 / 재정 / 화면	제공

완성 화면 살펴보기

- 「반복 실험」에서 회차별 결과와 두 시나리오의 차이를 확인한다.



실습 순서

- 작은 계산부터 확인한 뒤 여러 번 실행하고 결과를 비교한다.

작성할 기능	확인할 것
실행 준비	기존 도시 / 파일
① 계산 검사 / ② 방문 인원	통과 여부 / 같은 입력
③ 한 회차 실행 / ④ 결과 요약	초기화 / 평균과 변동성
⑤ 반복 비교 / ⑥ 용량별 비교	같은 입력 / 용량의 영향
통합 검사	저장 / 사용성 / 시연

기존 병원 모형과 연결하기

- 지난주 완성한 advance_queue()로 진료 인원과 남은 대기를 계산한다.
- 이번에는 시간별 방문 인원을 바꾸며 같은 계산을 여러 번 실행한다.
  - 도로 / 전력 / 급수 조건은 고정하고, 확충 비용은 계산하지 않는다.

이번 실험의 조건

- 매 회차 대기 0명에서 시작해 6시간 후 남은 인원을 비교한다.

항목	설정
방문 인원	매시간 6~14명 중 하나
기준 / 비교 용량	8명/시간, 10명/시간
반복 실험	시드 101~130, 총 30회
비교 지표	6시간 후 대기 인원(명)

실습 1 : 계산 검사

- `is_valid_step()` 함수를 작성한다.
  - 기존 계산 함수가 내놓은 답을 검사하는 함수다.
  - 진료 인원과 남은 대기가 규칙에 맞으면 `True`를 반환한다.
  - 인원수가 틀리거나 범위를 벗어나면 `False`를 반환한다.

사람 수와 진료 규칙

- 진료 인원은 대기 중인 전체 인원과 진료 용량 중 작은 값이다.

기존 대기 / 방문 / 용량	결과	판정
10 / 10 / 6	진료 6명 / 대기 14명	맞음
2 / 3 / 10	진료 5명 / 대기 0명	맞음
2 / 3 / 10	진료 4명 / 대기 1명	틀림
0 / 0 / 0	진료 0명 / 대기 0명	맞음

직접 해보기 : 계산 검사

- `validation_rules.py`의 TODO 1을 완성해 보자.
  - `python checks_validation.py --stage 1`
- 인원 합계는 같지만 진료 인원이 틀린 결과도 찾아야 한다.

계산 검사 해설

- 진료 규칙과 인원 보존이 모두 맞는지 확인한다.

```
served, remaining = result["served"], result["queue"]
if not all(nonnegative_integer(value) for value in (served, remaining)):
    return False

return served == min(queue + arrivals, capacity) and served + remaining == queue + arrivals
```

계산이 맞다는 것의 의미

- 검사를 통과하면 정한 규칙대로 계산했는지 확인할 수 있다.
- 실제 병원에 맞는 규칙인지는 실제 자료와 따로 비교해야 한다.
  - 이번 수업에서는 프로그램의 계산과 실험 절차를 확인한다.

실습 2 : 방문 인원 생성

- `make_arrivals()` 함수를 작성한다.
  - 정해진 시드로 시간별 방문 인원 리스트를 만든다.
  - 각 값은 6명부터 14명 사이의 정수다.
- 같은 환경에서 같은 시드와 조건을 주면 같은 리스트가 나와야 한다.

시드와 방문 인원

- 시드를 바꾸면 비교에 사용할 방문 인원도 바뀐다.

시드	1~6시간 방문 인원	다시 실행했을 때
101	9 / 14 / 11 / 13 / 6 / 14명	같은 리스트
102	8 / 11 / 8 / 14 / 8 / 12명	같은 리스트

직접 해보기 : 방문 인원 생성

- `validation_rules.py`의 TODO 2를 완성해 보자.
  - `python checks_validation.py --stage 2`
- 「방문 인원 생성」을 두 번 눌러 리스트를 비교한다.

방문 인원 생성 해설

- 이 실험에서만 사용할 난수 생성기를 만든다.

```
generator = random.Random(seed)
arrivals = []

for _ in range(hours):
    arrivals.append(generator.randint(low, high))

return arrivals
```

재현할 입력 확인

- 시드 101로 만든 방문 인원을 적어 둔다.
- 시드 102로 바꿨다가 101로 되돌려 같은 리스트인지 확인한다.
- 기존 도시 탭의 상태가 바뀌지 않았는지도 확인한다.

실습 3 : 한 회차 실행

- `run_scenario()` 함수를 작성한다.
  - 대기 0명에서 시작해 방문 인원을 시간순으로 처리한다.
  - 기존 `advance_queue()`를 호출하고 결과를 검사한다.
  - 시간별 기록과 마지막 대기 인원을 반환한다.

6시간 동안 바뀌는 대기

- 시드 101의 방문 인원과 용량 8명/시간으로 계산한다.

시간	방문 / 진료	남은 대기
1시간	9명 / 8명	1명
2시간	14명 / 8명	7명
3시간	11명 / 8명	10명
4시간	13명 / 8명	15명
5시간	6명 / 8명	13명
6시간	14명 / 8명	19명

직접 해보기 : 한 회차 실행

- `validation_rules.py`의 TODO 3을 완성해 보자.
  - `python checks_validation.py --stage 3`
- 「1회 실행」 후 마지막 대기가 19명인지 확인한다.

한 회차 실행 해설

- 대기와 기록은 함수 안에서 새로 만든다.

```
queue = 0
history = []

for hour, visitors in enumerate(arrivals, 1):
    result = advance_queue(queue, visitors, capacity)

    if not is_valid_step(queue, visitors, capacity, result):
        raise ValueError(f"{hour}시간째 대기열 계산이 규칙과 맞지 않습니다.")

    queue = result["queue"]
    history.append(
        {"hour": hour, "arrivals": visitors, "served": result["served"], "queue": queue}
    )
```

초기화와 극단적인 입력

- 같은 입력으로 다시 실행해도 앞 회차의 대기가 남으면 안 된다.

실행 조건	기대 결과
시드 101 / 용량 8을 두 번 실행	마지막 대기 19명으로 동일
방문 [0, 0] / 용량 8	대기 0명
방문 [3, 4] / 용량 0	마지막 대기 7명
방문 [3, 4] / 용량 100	마지막 대기 0명

실습 4 : 결과 요약

- `summarize()` 함수를 작성한다.
  - 회차별 마지막 대기 인원으로 평균과 표준편차를 구한다.
  - 회차 수, 최솟값, 최댓값도 함께 반환한다.
- 값이 없을 때와 대기 인원이 0명일 때를 구분한다.

평균과 자료 개수

- 평균이 같아도 실행마다 달라지는 정도는 다를 수 있다.

입력	평균	표준편차
`[8, 10, 12]`	10명	2명
`[2, 10, 18]`	10명	8명
`[10]`	10명	계산하지 않음
`[]`	계산하지 않음	계산하지 않음

직접 해보기 : 결과 요약

- `validation_rules.py`의 TODO 4를 완성해 보자.
  - `python checks_validation.py --stage 4`
- 빈 목록과 값이 한 개인 목록도 검사한다.

결과 요약 해설

- 자료 개수에 따라 계산 가능한 요약값을 반환한다.

```
count = len(values)

if count == 0:
    return {"count": 0, "mean": None, "stddev": None, "minimum": None, "maximum": None}

mean = sum(values) / count
# 표본 표준편차는 두 개 이상의 결과가 있어야 계산할 수 있다.
stddev = statistics.stdev(values) if count >= 2 else None

return {
    "count": count,
    "mean": mean,
    "stddev": stddev,
    "minimum": min(values),
    "maximum": max(values),
}
```

요약값 확인

- 모든 회차의 결과를 사용하고 평균과 범위를 함께 읽는다.

기준 시나리오 30회	값
평균	12.60명
표본 표준편차	약 5.47명
최솟값 / 최댓값	2명 / 22명

실습 5 : 반복 비교

- `compare_scenarios()` 함수를 작성한다.
  - 시드마다 방문 인원 리스트를 한 번 만든다.
  - 같은 리스트로 기준 용량과 비교 용량을 각각 실행한다.
  - 두 결과와 줄어든 대기 인원을 기록한다.

한 회차의 두 시나리오

- 같은 방문 인원을 두 용량에 공통으로 사용한다.

항목	비교 조건
공통 방문 인원	9 / 14 / 11 / 13 / 6 / 14명
초기 대기	두 시나리오 모두 0명
기준 8명/시간	6시간 후 대기 19명
비교 10명/시간	6시간 후 대기 8명

직접 해보기 : 반복 비교

- `validation_rules.py`의 TODO 5를 완성해 보자.
  - `python checks_validation.py --stage 5`
- 「반복 비교」로 시드 101~130의 결과를 확인한다.

반복 비교 해설

- 방문 인원을 한 번 만들고 두 시나리오에 공통으로 사용한다.

```
for seed in seeds:
    arrivals = make_arrivals(seed, hours, low, high)
    baseline = run_scenario(arrivals, baseline_capacity)
    policy = run_scenario(arrivals, policy_capacity)

    results.append(
        {
            "seed": seed,
            "arrivals": arrivals,
            "baseline": baseline,
            "policy": policy,
            "reduction": baseline["final_queue"] - policy["final_queue"],
        }
    )
```

두 결과와 변동성 확인

- 평균 대기는 줄었지만, 모든 실행에서 대기가 0명이 된 것은 아니다.

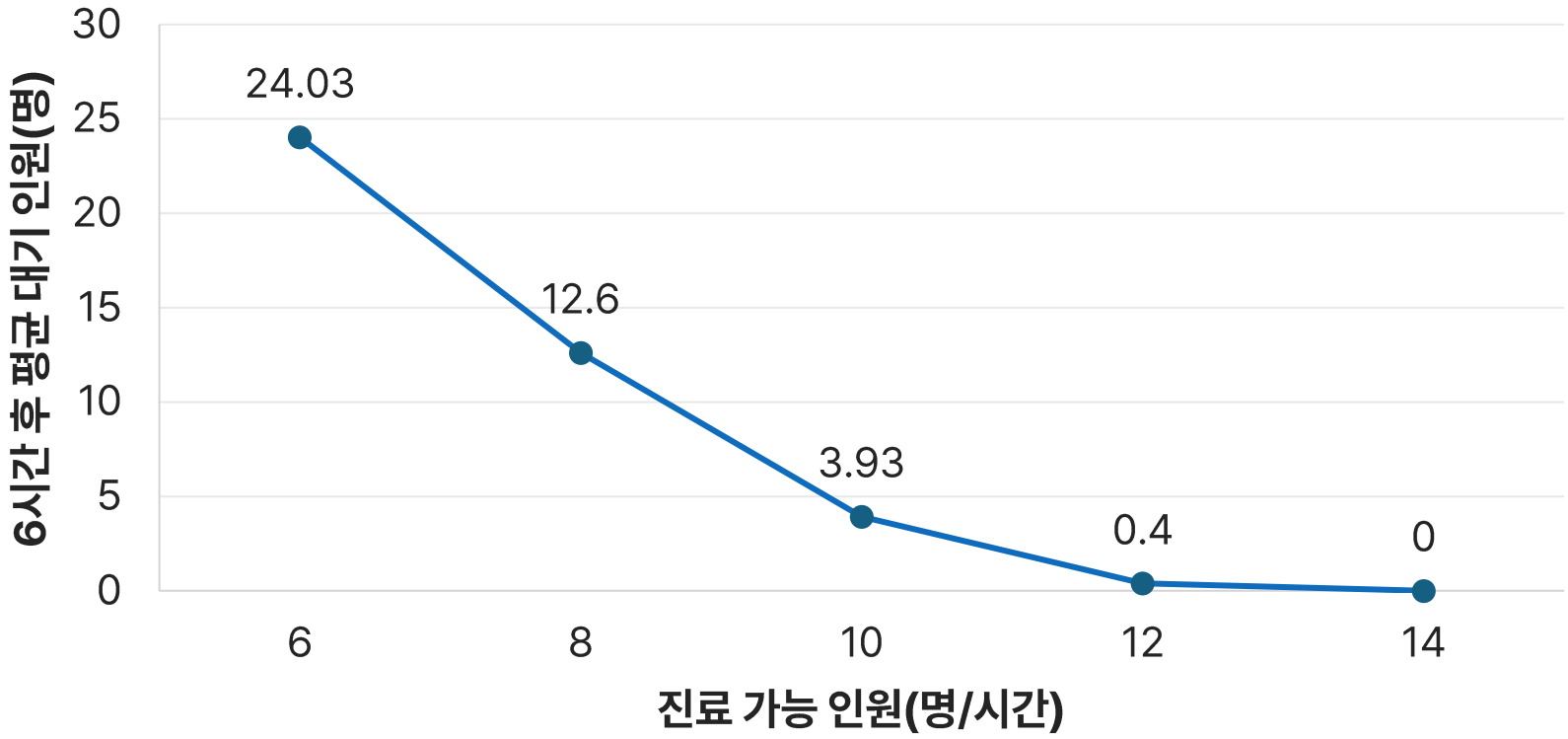
30회 결과	기준 8명/시간	비교 10명/시간
6시간 후 평균 대기	12.60명	3.93명
표본 표준편차	약 5.47명	약 3.18명
최솟값 / 최댓값	2명 / 22명	0명 / 10명

실습 6 : 민감도 분석

- `sensitivity_analysis()` 함수를 작성한다.
  - 병원 용량을 6, 8, 10, 12, 14명/시간으로 바꿔 실행한다.
  - 각 용량에 같은 30회분 방문 인원을 사용한다.
  - 용량별 결과와 요약값을 반환한다.

병원 용량과 평균 대기

- 같은 방문 인원에서 용량을 늘렸을 때의 변화를 비교한다.



직접 해보기 : 민감도 분석

- `validation_rules.py`의 TODO 6을 완성해 보자.
  - `python checks_validation.py --stage 6`
- 「용량별 비교」 표에서 평균과 최대 대기를 확인한다.

민감도 분석 해설

- 같은 방문 인원으로 용량별 대기 인원을 구하고 요약한다.

```
inputs = []

for seed in seeds:
    inputs.append(make_arrivals(seed, hours, low, high))

results = []

for capacity in capacities:
    require_integer(capacity, "진료 용량")

    values = []
    for arrivals in inputs:
        values.append(run_scenario(arrivals, capacity)["final_queue"])
    results.append({"capacity": capacity, "values": values, "summary": summarize(values)})

return results
```

조건을 바꿔 해석하기

- 방문 인원의 최댓값을 바꾸고 용량별 결과를 다시 확인한다.
- 용량 14명/시간이 항상 가장 좋은 정책이라고 말할 수 있을까?
  - 방문 가정, 비용, 급수 조건을 포함했는지 함께 살펴본다.
  - 병원이 큰 곳에 대기가 많아도, 방문 인원이 많은 탓일 수 있다.

시연 순서와 최종본 준비

- 예제 실행, 조건 변경, 결과 비교, 저장 순서로 짧게 시연한다.
- 확인한 코드와 예제 / 설정 / 실험 기록을 한 폴더에 보관한다.
- 코드 동결 뒤 중요한 오류를 고쳤다면 관련 검사와 시연을 다시 한다.

정리

- 작은 예제로 계산을 검사하고, 같은 입력을 다시 만들었다.
- 매 회차 초기화한 뒤 같은 입력으로 두 시나리오를 비교했다.
- 평균 / 변동성 / 실험 조건을 함께 기록하고, 통합 실행을 확인했다.

감사합니다.

utilForever@gmail.com

https://github.com/utilForever

Facebook, Twitter, Instagram, LinkedIn: utilForever